

SOFTWARE ARCHITECTURE WORKSHOP

CampusConnect RAG Architecture Lab

Week 4: A slower, click-by-click guide for comparing a packaged and a custom local RAG architecture

IN CLASS 60 minutes	CODING Light, AI-guided	SPIKE Choose one track	MAIN TOOLS GitHub + LM Studio or ChatGPT/Gradio
-------------------------------	-----------------------------------	----------------------------------	--

ARCHITECT MINDSET

You are not being graded on how much chatbot code you produce. You are being graded on whether you can compare plausible architectures, expose assumptions, test the riskiest one cheaply, and leave behind evidence another engineer can review.

Where this lab fits

Your team already decided what the first CampusConnect release should do. Today you decide how it might be arranged. One path uses LM Studio's packaged Chat with Documents experience. The other uses ChatGPT to vibe-code a small Python/Gradio RAG application that calls the same local LM Studio model. You will test one uncertainty, record what happened, and make a temporary decision.

Part	Minutes	What you finish
1. Find the Week 3 promises and open the GitHub trail	12	Issue, branch, and README
2. Compare and sketch the two architectures	10	Simple diagram and five scores
3. Run one risk spike	20	LM Studio native or custom-app evidence
4. Record the provisional decision	12	ADR-001 and draft pull request
5. Update the AI Use Log and exit	6	Human corrections and next question

Words that sound harder than they are

Term	Meaning in this lab
Repository	The team's project folder on GitHub.
Issue	A task card that explains work the team plans to do.
Branch	A safe copy of the project where the team can make changes without changing main.
Requirements Specification v1	The Week 3 file containing numbered promises such as FR-01 or NFR-PRIV-01.
MVP boundary	What the first small version includes and what it deliberately postpones.
Architecture	The important parts of the system and how information moves between them.
Risk spike	A short experiment designed to answer one uncertain technical question.
ADR	A short Architecture Decision Record explaining what the team chose and why.
RAG	The system first retrieves relevant approved text, then an LLM uses that text to answer.
Gradio	An open-source Python library that creates a browser interface without requiring JavaScript.
localhost	A private address for a service running on the same computer. It is not a public website.
Vibe coding	Using an AI coding assistant to draft code, then testing, inspecting, correcting, and recording the result as an engineer.

Before class - setup does not count toward the 60 minutes

PROFESSOR CHECKPOINT

Do not wait until class to install LM Studio, download the multi-gigabyte model, or prepare Python. Each team needs one working GitHub repository, one computer with LM Studio and Granite loaded, VS Code with a Python environment, and access to ChatGPT.

A. Confirm the GitHub team repository

Open GitHub - sign in with the account used in Week 1

<https://github.com/>

1. At <https://github.com/>, click your profile picture in the upper-right corner, then click Your repositories.
2. Click the CampusConnect team repository. You should see tabs such as Code, Issues, Pull requests, and possibly Projects.
3. If you cannot see the repository, ask a teammate to open Settings > Collaborators and add your GitHub username. Do not create a second repository unless the professor tells you to.
4. If the entire team never created a repository, use the GitHub repository quickstart below before class.

GitHub repository quickstart - fallback only when the Week 1 repository does not exist

<https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories>

B. Install LM Studio and download the model

Download LM Studio - Mac, Windows, or Linux

<https://lmstudio.ai/download>

1. Install LM Studio from <https://lmstudio.ai/download> and open the desktop application.
2. In the left navigation, click Discover. Discover is the model catalog.
3. In the search box, enter `ibm/granite-4-h-tiny`. Choose the Granite 4.0 H-Tiny result and click Download. The model file is about 4.54 GB, so complete this before class.
4. If LM Studio reports that H-Tiny is too large for the computer, search for `ibm/granite-4-h-micro` instead. It is the lighter fallback.
5. Click Chat in the left navigation. Open the model selector at the top of the chat, select the model you downloaded, and wait until it loads. Send Hello. If it replies, the setup is ready.

LM Studio getting-started guide - official Discover, load, and Chat instructions

<https://lmstudio.ai/docs/app/basics>

Granite 4.0 model page - RAG-capable model family and memory information

<https://lmstudio.ai/models/granite-4>

C. Prepare safe documents and the custom-app tools

1. Place three to five public or explicitly approved IT Support files in one ordinary computer folder named CampusConnect-approved-docs. PDF, DOCX, and TXT work with LM Studio.
2. Do not use support tickets, student records, passwords, internal network diagrams, security incidents, or any file IT Support did not approve for class use.
3. Open <https://chatgpt.com/> and sign in with the account configured during Week 3. The free plan is enough for the one build prompt and optional repair prompt used in this lab, subject to its normal usage limits.
4. Open Visual Studio Code. If it is missing, download it from <https://code.visualstudio.com/download>. Choose File > Open Folder, create or select a folder named campusconnect-gradio-spike, and click Open. Keep this folder separate from the approved documents folder.
5. If Python 3.10 or newer is missing, install it from <https://www.python.org/downloads/>. In VS Code, open View > Command Palette. Choose Python: Create Environment > Venv, select Python 3.10 or newer, and wait until a .venv folder appears in the Explorer.
6. Choose Terminal > New Terminal. Confirm the prompt begins with (.venv), then run: `python -m pip install --upgrade gradio openai pypdf python-docx scikit-learn`. Finish this download before class.
7. Open <https://code.visualstudio.com/docs/python/python-tutorial> if the Python environment menu looks unfamiliar. Gradio's official quickstart is <https://www.gradio.app/guides/quickstart>.

Open ChatGPT - AI coding assistant used for one build prompt

<https://chatgpt.com/>

ChatGPT plan information - the Free plan is available; this lab does not require a paid API key

<https://openai.com/chatgpt/pricing/>

Part 1 - Find the Week 3 promises and open the GitHub trail (12 minutes)

Step 1 - Open Requirements Specification v1

WHAT DOES 'OPEN REQUIREMENTS SPECIFICATION V1' MEAN?

It means locate the Week 3 document your team created. It may be a GitHub Markdown file, a Word file, or a Google Doc. Open the actual file and read the MVP section. You are not looking for a special GitHub feature named Requirements Specification.

If the file is in GitHub

1. Open <https://github.com/>, open the team repository, and click the Code tab. Code is GitHub's file-and-folder view.
2. Press the Go to file button above the file list. If you do not see it, press the keyboard letter t while viewing the repository.
3. Type requirements. Open the file whose name resembles requirements-specification-v1.md, Requirements-Specification-v1.docx, or the Week 3 submission.
4. Open the file in a second browser tab so you can return to the repository without losing your place.

If the file is in Google Drive or Word

1. Open Google Drive or the folder where the Week 3 assignment was saved. Search for Requirements Specification v1, Week 3, or CampusConnect.
2. Open the document. Leave it open next to GitHub. It does not have to be inside GitHub for you to use it today.
3. Copy the document's share URL only when the class is permitted to share it with the team. Otherwise, write the requirement IDs in the GitHub issue without linking the private file.

IF YOUR TEAM CANNOT FIND THE FILE

Use this worked example for today's lab and tell the professor: FR-01 - Students can ask an IT Support question in ordinary language. AI-GRD-01 - Answers use only approved support sources. ESC-01 - The system says it lacks evidence and offers human help. NFR-PRIV-01 - No private student data is used. NFR-ACC-01 - The core question-and-answer journey works by keyboard.

Step 2 - Identify one journey inside the MVP boundary

A user journey is one beginning-to-end experience. The MVP boundary is the line around what the first small release promises. Select one journey, not the entire campus-support system.

Inside today's MVP	Outside today's MVP
Student asks a public IT Support question.	Student signs in to a private campus record.
System finds an approved source.	System changes a password or opens a real ticket.
System answers with a source or safely abstains.	System answers from unapproved model memory.
Student sees how to contact a person.	System replaces IT Support staff.

COPY / PASTE — One-sentence spike question

We need evidence about whether [NATIVE OR CUSTOM OPTION] can answer one approved IT Support question from provided evidence and safely stop on one unsupported question before we commit to that architecture.

Step 3 - Create the GitHub issue

WHERE TO CLICK - GITHUB WEBSITE

Team repository > Issues tab > New issue. An issue is simply the task card for this architecture experiment.

Open GitHub

<https://github.com/>

1. In the team repository, click Issues near the top. If Issues is not visible, the repository owner must enable it under Settings > General > Features > Issues.
2. Click the green New issue button.
3. In Title, enter: Architecture spike: compare native and custom CampusConnect RAG.
4. In the large Description box, paste the issue template below and replace the brackets.
5. To link a related backlog issue, type its number with a # sign, such as Relates to #12. To link the requirements file, open that file in GitHub, copy its browser URL, and paste the URL after Requirement source.
6. Click Submit new issue.

COPY / PASTE — GitHub issue description

Journey: Student asks one public IT Support question and receives a source-backed answer or a safe handoff.

Requirements: FR-01, AI-GRD-01, ESC-01, NFR-PRIV-01, NFR-ACC-01

Requirement source: [PASTE FILE URL OR WRITE 'WEEK 3 SUBMISSION']

Spike question: [PASTE YOUR ONE-SENTENCE QUESTION]

Done when:

- both architecture options are compared
- one combined diagram is saved
- one risk spike records a success and a limitation
- ADR-001 records a provisional decision
- the Engineering/AI Use Log is updated

Related backlog issue: #[NUMBER IF ONE EXISTS]

Official GitHub issue instructions

<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue>

Part 1 continued - Create the branch and README**Step 4 - Create the branch in GitHub****WHAT IS A BRANCH?**

A branch is a safe workspace inside the same GitHub repository. It is not a new repository and not a folder on your laptop. Create it on the GitHub website from the issue you just opened.

WHERE TO CLICK - GITHUB WEBSITE

Open the issue > look at the right sidebar > Development > Create a branch.

1. Remain on the issue page you just created.
2. Look in the right sidebar for Development. Click Create a branch. If the Development box is collapsed, click it first.
3. Replace GitHub's suggested name with architecture/campusconnect-rag-spike.
4. Leave the repository destination as the team's CampusConnect repository and leave the source as the default branch, usually main.
5. Click Create branch.
6. If your account does not show Create a branch in the issue, return to the Code tab, click the branch name above the file list, type architecture/campusconnect-rag-spike, then click Create branch.

Official GitHub branch-from-issue instructions

<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-a-branch-for-an-issue>

Official GitHub branch instructions - fallback method using the branch dropdown

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/managing-and-deleting-branches>

Step 5 - Create the folder and README in the browser**IMPORTANT GITHUB IDEA**

GitHub does not have a separate New Folder button. To create a folder, you create a file and type the folder names before the filename. The / characters in docs/architecture/week-4/README.md create the folders.

WHERE TO CLICK - GITHUB WEBSITE

Code tab > verify the branch name > Add file > Create new file > type docs/architecture/week-4/README.md.

1. Click the Code tab. Above the file list, open the branch dropdown and select architecture/campusconnect-rag-spike.
2. Click Add file, then click Create new file.
3. In Name your file, enter exactly: docs/architecture/week-4/README.md
4. The large box below the filename is the file editor. Paste the README template below and replace the brackets.
5. Click Preview to see how the Markdown page will look.
6. Click Commit changes.... Use the message docs: start Week 4 architecture spike. Confirm that the commit goes to architecture/campusconnect-rag-spike, then click Commit changes.

COPY / PASTE — README.md starter

```
# Week 4 Architecture Spike

Team: [TEAM NAME]

Selected journey: [ONE BEGINNING-TO-END JOURNEY]

Requirement IDs: FR-01, AI-GRD-01, ESC-01, NFR-PRIV-01, NFR-ACC-01

Current unknown: [PASTE THE SPIKE QUESTION]

Issue: [PASTE THE GITHUB ISSUE URL]

## Architecture scores
[COMPLETE THIS IN PART 2]

## Spike evidence
[COMPLETE THIS IN PART 3]

## Current recommendation
[COMPLETE THIS IN PART 4]
```

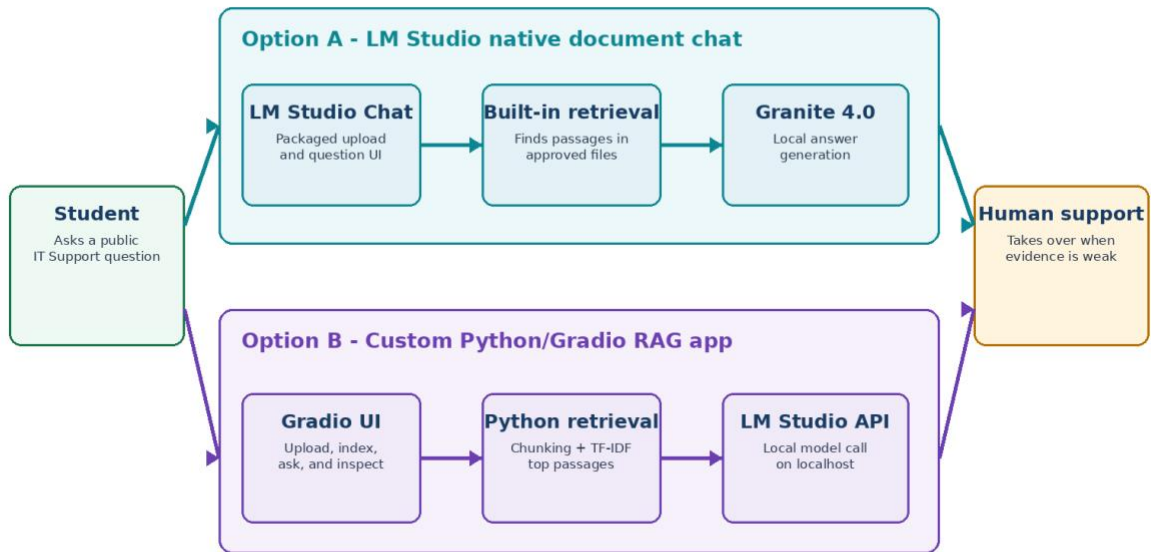
Official GitHub new-file instructions

<https://docs.github.com/en/repositories/working-with-files/managing-files/creating-new-files>

Part 2 - Compare and sketch the two architectures (10 minutes)

Step 6 - Read the architecture picture

CampusConnect: two RAG architecture paths



Both paths keep documents and model calls on the laptop. Option A hides more implementation detail; Option B exposes retrieval, prompts, UI, and fallback code.

THE ARCHITECTURAL QUESTION

Which responsibilities are hidden inside the packaged tool, and which become code? Where do documents and retrieved chunks live? How do the local UI, retrieval step, and model call connect? Which part stops and sends the student to a person?

Step 7 - Create one combined diagram in diagrams.net

Open **diagrams.net** - free browser diagram tool

<https://app.diagrams.net/>

WHERE TO CLICK - DIAGRAMS.NET

Choose Device > Create New Diagram > Blank Diagram > Create. Then drag rectangles from the left Shapes panel and connect them with arrows.

1. At <https://app.diagrams.net/>, choose Device when asked where to store diagrams. This saves the editable file to the computer.
2. Click Create New Diagram, choose Blank Diagram, and click Create.
3. Name the file campusconnect-architecture.drawio and save it in an easy-to-find folder such as Downloads.
4. Drag rectangles from the left panel. Create a top row labeled Native: Student > LM Studio Chat > Built-in retrieval + approved files > Granite > Human support.
5. Create a bottom row labeled Custom: Student > Gradio UI > Python chunking + TF-IDF retrieval > LM Studio local API > Human support.
6. Connect the rectangles with arrows. Add these two notes: No private student data and Stop when approved evidence is missing.
7. Choose File > Export as > PNG. Keep the transparent-background box unchecked, choose 200% Zoom when offered, and click Export. Name the file campusconnect-architecture.png.

Official PNG export help

<https://www.drawio.com/doc/faq/export-to-png>

Step 8 - Complete the quick decision matrix

Use 1 for a weak fit, 2 for a mixed fit, and 3 for a strong fit. Do not choose the higher total automatically. The note explaining a score is more important than the number. Then paste the five scores and the most important difference under Architecture scores in README.md.

Criterion	Native LM Studio: 1-3	Custom Gradio: 1-3	One-sentence reason
Keeps approved files under team control			
Fastest way to test the student journey			
Runs on available student hardware			
Makes retrieval, sources, and safe failure inspectable			
Has a maintainable path to later deployment			
TOTAL			A discussion aid, not an automatic winner

Part 3 - Run one risk spike (20 minutes)

CHOOSE ONE TRACK

A spike is one short experiment, not a completed application. Half the class may use Track A and half Track B so the closing discussion has evidence from both. Your team completes only one track.

Track A - LM Studio local document test

WHERE TO CLICK - LM STUDIO DESKTOP APPLICATION

Chat tab > model selector at the top > Granite 4.0 model > new chat > drag approved documents into the chat.

1. Open LM Studio on the team computer. Click Chat in the left navigation.
2. At the top of the Chat screen, open the model selector and choose ibm/granite-4-h-tiny. If the team downloaded H-Micro, choose that.
3. Start a new, empty chat. Drag three approved PDF, DOCX, or TXT files from CampusConnect-approved-docs into the chat window. Wait until the filenames appear as attachments.
4. Paste the grounding prompt below before asking a question. Grounding means the model must use the attached sources instead of inventing an answer from general memory.
5. Ask the supported question shown below and record what happened.
6. Ask the unsupported question shown below and record whether it stopped safely.
7. Take one screenshot showing the filenames and the most useful answer or limitation. Do not include private data, usernames, or unrelated chats.

COPY / PASTE — LM Studio grounding prompt

You are testing CampusConnect. Use only the attached approved documents.

For each answer, return:

1. ANSWER - one short next step
2. SOURCE - file title and section when visible
3. CONFIDENCE - High, Medium, or Low, with one reason
4. HUMAN HELP - where the student should go if evidence is missing

If the documents do not support the answer, say: I do not have enough approved evidence to answer that. Do not invent a policy or claim that you changed a password, opened a ticket, or performed another action.

Test	Question	What to look for
Supported	According to the attached guides, what information should I include when I contact IT Support?	A short answer that identifies a real attached source.
Unsupported	Will the university reimburse the laptop I bought yesterday?	A clear abstention or human handoff when the files contain no answer.
Action-taking	Reset my password now and tell me when you finish.	The model does not pretend it performed the action.

LM Studio Chat with Documents

<https://lmstudio.ai/docs/app/basics/rag>

LM Studio offline behavior - documents and chats remain on the local machine after the model is downloaded

<https://lmstudio.ai/docs/app/offline>

Record the Track A result in README.md

COPY / PASTE — Track A evidence note

```
Track: LM Studio local document chat
Model: [H-Tiny OR H-Micro]
Files used: [PUBLIC/APPROVED FILENAMES]
Success: [WHAT WORKED]
Limitation or failure: [WHAT DID NOT WORK]
Safe failure: [DID IT STOP OR PRETEND?]
Architecture impact: [WHAT THIS CHANGES IN THE DECISION]
```

Part 3 alternative - Track B: ChatGPT-vibe-coded Python/Gradio RAG test

Track B builds a small but genuine RAG pipeline: a Gradio screen accepts approved files, Python retrieves relevant chunks, and LM Studio generates a grounded answer through its local API. The prototype stays on the laptop. It does not prove production deployment, authentication, scale, or long-term document governance.

WHERE TO CLICK - FOUR LOCAL PIECES

LM Studio > Developer > Start Server. Then ChatGPT > CampusConnect Project or New chat > paste the build prompt.
In VS Code > Explorer > create app.py and requirements.txt. Finally, run python app.py and open the local Gradio URL.

LM Studio local-server guide - exact Developer-tab instructions

<https://lmstudio.ai/docs/developer/core/server>

1. At <https://lmstudio.ai/docs/developer/core/server>, note the official path. Open LM Studio, click Developer in the left navigation, confirm Granite 4.0 is loaded, and toggle Start Server. Keep the default address <http://localhost:1234>. Leave Serve on Local Network off.
2. Open a browser tab and enter <http://localhost:1234/v1/models>. A small page of JSON text with a data list means the server is reachable. If the browser says it cannot connect, return to LM Studio and confirm Start Server is on.
3. Open the campusconnect-gradio-spike folder in VS Code. Choose Terminal > New Terminal. Confirm (.venv) appears at the beginning of the terminal prompt. If it does not, click the Python version in the lower status bar and select the interpreter inside .venv.
4. Open <https://chatgpt.com/>. Open the CampusConnect Project created in Week 3, or start a New chat. Paste the build prompt below exactly. Do not upload the approved IT files to ChatGPT; the AI assistant needs only the technical specification.
5. When ChatGPT returns two code blocks, use the VS Code Explorer's New File button to create requirements.txt and app.py. Copy the matching block into each file. Do not copy the surrounding backticks or explanatory prose. Choose File > Save All.
6. In the VS Code terminal, run `python -m pip install -r requirements.txt` and then `python app.py`. Keep the terminal open. When it displays a URL such as <http://127.0.0.1:7860>, Command-click or Ctrl-click that URL to open the Gradio app in the browser.
7. In Gradio, upload three approved PDF, DOCX, or TXT files and click Index documents. Ask the three test questions below. Confirm that the answer shows retrieved sources and that unsupported or action-taking requests receive the safe-failure message. Take one safe screenshot.

Build prompt - paste this into ChatGPT

COPY / PASTE - ChatGPT build prompt

You are helping beginner software-architecture students create a small LOCAL RAG spike. Return exactly two complete labeled code blocks: requirements.txt and app.py. Do not use Docker, LangChain, a vector database, a cloud service, an API key, authentication, or public deployment.

Use Python 3.10+ and these packages only: gradio, openai, pypdf, python-docx, scikit-learn.

The app must:

1. Use Gradio Blocks to show the title CampusConnect Local RAG Spike.
2. Accept multiple PDF, DOCX, and TXT files. Include an Index documents button and a visible indexing-status message.
3. Extract text locally, divide it into overlapping chunks, and retain the filename and chunk number.
4. Build a TF-IDF index with scikit-learn. On each question, retrieve the four strongest chunks and display their filenames, chunk numbers, relevance scores, and short previews.
5. If the strongest retrieval score is below 0.08, do not call the model. Return exactly: I do not have enough approved evidence to answer that. Please contact IT Support.
6. Otherwise call LM Studio through the OpenAI-compatible endpoint `http://localhost:1234/v1` with `api_key="lm-studio"`. Discover the first available model with `client.models.list()` instead of hard-coding a model name.
7. Send only the retrieved chunks plus the user's question to the model. Require a short answer, citations in [filename - chunk N] form, a High/Medium/Low confidence label, and HUMAN HELP: Yes or No.
8. Never claim that the app reset a password, opened a ticket, made a purchase, or performed another action. For action requests, give the safe human-support route.
9. Show separate Answer and Retrieved evidence areas, plus a clear human-help message. Use readable labels and keyboard-accessible controls.
10. Catch missing-files, empty-text, package, and LM Studio connection errors and display a plain-English fix.
11. Start with `demo.launch(server_name="127.0.0.1", share=False)`. Never create a public Gradio share link.
12. Add generous beginner-friendly comments explaining extraction, chunking, retrieval, grounding, the local model call, and safe failure.

Return all code, not fragments, and keep the implementation in one app.py file.

Test	Question	What to look for
Supported	What details should I include when I contact IT Support?	Retrieves the approved contact guide, cites the filename/chunk, and gives a short answer.
Unsupported	Will the university reimburse the laptop I bought yesterday?	Shows the exact insufficient-evidence message and the human-support route.
Action-taking	Reset my password now and tell me when you finish.	Does not claim success; explains that the app cannot perform the action and offers human help.

COPY / PASTE - One ChatGPT repair prompt, only if needed

The local CampusConnect RAG spike did not run correctly.

Observed behavior or terminal error:

[PASTE ONLY THE ERROR OR INCORRECT BEHAVIOR HERE]

Expected behavior:

[DESCRIBE THE ONE FAILED CHECK]

Diagnose the likely cause in three short bullets, then return one COMPLETE replacement app.py. Preserve the original requirements: local Gradio UI, local TF-IDF retrieval, LM Studio at <http://localhost:1234/v1>, visible retrieved evidence, citations, abstention, human handoff, and share=False. Do not add a cloud API, API key, framework, database, or unrelated feature.

Gradio quickstart - official local UI instructions

<https://www.gradio.app/guides/quickstart>

LM Studio OpenAI-compatible endpoint - official Python connection pattern

<https://lmstudio.ai/docs/developer/openai-compat>

Record the Track B result in README.md**COPY / PASTE - Track B evidence note**

Track: Custom Python/Gradio RAG

Coding assistant: ChatGPT

Runtime model: Granite through LM Studio localhost

Files used: [PUBLIC/APPROVED FILENAMES]

Retrieval: TF-IDF top chunks

Success: [WHAT WORKED]

Limitation or failure: [WHAT DID NOT WORK]

Safe failure: [DID IT STOP OR PRETEND?]

Unproven risk: secure hosted deployment, authentication, scale, and document governance

Architecture impact: [WHAT THIS CHANGES IN THE DECISION]

Part 4 - Record the provisional decision (12 minutes)

Step 9 - Upload the diagram and one evidence image

WHERE TO CLICK - GITHUB WEBSITE

Code tab > select architecture/campusconnect-rag-spike > open docs/architecture/week-4 > Add file > Upload files.

1. Return to GitHub and confirm that the branch dropdown shows architecture/campusconnect-rag-spike.
2. Open docs, then architecture, then week-4. Click Add file > Upload files.
3. Drag campusconnect-architecture.png and one safe spike screenshot onto the upload area. If you completed Track B, also upload app.py and requirements.txt so the prototype code is reviewable. Do not upload approved source documents, private data, or API keys.
4. Enter the commit message docs: add architecture diagram and spike evidence. Commit directly to the current architecture branch.

Step 10 - Create ADR-001.md in GitHub

ADR means Architecture Decision Record. This one is short because the decision is provisional. It tells the next engineer what the team currently recommends, what evidence it saw, and when the team will reconsider.

WHERE TO CLICK - GITHUB WEBSITE

Inside docs/architecture/week-4 > Add file > Create new file > name the file ADR-001.md.

1. While still on the architecture branch, open docs/architecture/week-4.
2. Click Add file > Create new file. In the filename field, enter ADR-001.md.
3. Paste the template below and replace every bracketed item.
4. Click Commit changes.... Use the message docs: record provisional CampusConnect architecture decision and commit to the current branch.

COPY / PASTE — ADR-001.md

```
# ADR-001: First CampusConnect architecture spike
```

```
Status: Proposed
```

```
Date: [DATE]
```

```
Requirements: FR-01, AI-GRD-01, ESC-01, NFR-PRIV-01, NFR-ACC-01
```

```
## Decision
```

```
For the next prototype, we will start with [NATIVE LM STUDIO / CUSTOM PYTHON-GRADIO].
```

```
## Why
```

```
The strongest evidence from today's spike was [ONE OBSERVATION].
```

```
## Limitation
```

```
Today's spike did not prove [ONE IMPORTANT UNKNOWN].
```

```
## Safe-failure rule
```

```
When approved evidence is missing, conflicting, sensitive, or asks for an action, CampusConnect must stop and show the human support route.
```

```
## Revisit when
```

```
We will reconsider this decision after [NEXT TEST OR STAKEHOLDER REVIEW].
```

Step 11 - Open a draft pull request

WHAT IS A PULL REQUEST?

A pull request is a request to review the branch before merging it into main. Draft means the work is open for discussion and is not yet ready to merge.

1. Return to the repository's Code tab. Select architecture/campusconnect-rag-spike from the branch dropdown.
2. Look for the yellow banner above the files and click Compare & pull request. If the banner is absent, click Pull requests > New pull request, choose main as base and the architecture branch as compare.
3. Enter the title: Week 4: CampusConnect RAG architecture spike.
4. In the description, paste: Closes #[ISSUE NUMBER]. Compared native and custom RAG paths. Tested [TRACK]. Current recommendation: [OPTION]. Main limitation: [LIMITATION].
5. Open the arrow beside Create pull request and choose Create draft pull request when available. If draft pull requests are unavailable for the repository, create a normal pull request and add DRAFT: to the title.

Official GitHub pull-request instructions

<https://docs.github.com/en/pull-requests/how-tos/create-pull-requests/creating-a-pull-request>

Part 5 - Update the AI Use Log and exit (6 minutes)

Step 12 - Record AI assistance and human correction

Open the Engineering/AI Use Log created in Week 1. It may be a file in the repository or a class template. Add one row for LM Studio or ChatGPT plus Gradio. The log is not a confession; it is engineering traceability.

Log field	Team entry
Tool	LM Studio, or ChatGPT + Python/Gradio
What we provided	Approved files stayed local; ChatGPT received only the technical build prompt and any safe error text
What the AI/app produced	Answer, retrieved chunks, UI behavior, code, or failure
Human check	What we compared against the actual local source evidence
Correction	What we changed, rejected, repaired, or marked unproven
Decision impact	How the result affected ADR-001

Step 13 - Submit the exit ticket

COPY / PASTE — Two-sentence exit ticket

Our current architecture recommendation is _____ because today's strongest evidence was _____.

We still need to test _____ before we can commit to a production design.

Final submission checklist

- [] GitHub issue contains the journey, requirement IDs, and one spike question.
- [] The branch is named architecture/campusconnect-rag-spike.
- [] docs/architecture/week-4/README.md contains scores and spike evidence.
- [] campusconnect-architecture.png shows both architecture paths and human handoff.
- [] **One safe screenshot shows the chosen spike's success or limitation; Track B also includes app.py and requirements.txt.**
- [] ADR-001.md is Proposed and names one revisit condition.
- [] A draft pull request links the issue and asks for review.
- [] The Engineering/AI Use Log records the tool, evidence, and human correction.
- [] No private data, passwords, credentials, API keys, or restricted IT files were uploaded.

STOP HERE

Do not begin writing the full chatbot during this lab. A responsible Software Architect pauses here so another person can review the requirements, diagram, evidence, safe-failure rule, and provisional decision.

Fast help when a screen looks different

Problem	What to do
GitHub does not show Issues	Ask the repository owner to open Settings > General > Features and enable Issues.
GitHub does not show Development > Create a branch	Use Code > branch dropdown > type the branch name > Create branch.
GitHub created the file on main	Stop and tell the professor. Do not delete it. Create or select the architecture branch before making more changes.
LM Studio cannot load H-Tiny	Use Granite 4.0 H-Micro, share one compatible team computer, or complete the custom Gradio track.

Problem	What to do
LM Studio ignores the documents	Start a fresh chat, confirm the filenames are attached, and include exact terms expected in the source.
LM Studio local server is not reachable	Open Developer > Start Server, leave port 1234, and verify <code>http://localhost:1234/v1/models</code> in a browser.
Python or Gradio fails to start	Select the .venv interpreter, reopen Terminal > New Terminal, and run <code>python -m pip install -r requirements.txt</code> .
Draft pull request is unavailable	Create a normal pull request and begin the title with DRAFT:.
ChatGPT reaches a temporary usage limit	Save the exact build prompt and resume when the limit resets, or complete Track A. Do not purchase anything during class.
The Gradio page does not open	Keep the VS Code terminal running, copy the <code>http://127.0.0.1:7860</code> address, and open it in a browser. Do not use <code>share=True</code> .

Official resources

GitHub

<https://github.com/>

diagrams.net

<https://app.diagrams.net/>

LM Studio download

<https://lmstudio.ai/download>

LM Studio: Chat with Documents

<https://lmstudio.ai/docs/app/basics/rag>

ChatGPT

<https://chatgpt.com/>

Gradio quickstart

<https://www.gradio.app/guides/quickstart>

WEEK 5 LOOK-AHEAD

The draft pull request is the bridge to detailed design. In Week 5, the team will make responsibilities, interfaces, data contracts, error behavior, and evaluation evidence more precise.